

order invariant prior demo

Eric Chuu

3/20/2022

Parameter and Data Generation

In the following experiments, we have $\beta \in \mathbb{R}^{3 \times 2}$ and $\Omega \in \mathbb{R}^{3 \times 3}$.

```
set.seed(214)
m = 3      # number of columns in y, number of rows in beta
k0 = 2     # number of columns in beta (number of factors)
n = 500    # number of rows in y (number of observations)
```

Below, we generate the true parameters. We take $\Omega = I_m$ and randomly generate β column by column. Then, the data y is sampled from a MVN distribution with mean 0 and covariance matrix $\Sigma = \Omega + \beta\beta'$.

```
tau = 0.25
omega0 = tau * diag(1, m) # identity matrix
beta0 = matrix(0, m, k0)
beta0[,1] = c(rtnorm(n = 1, lower = 0, mean = -1), rnorm(n = m - 1, mean = 1))
beta0[,2] = c(0, rtnorm(n = 1, lower = 0, mean = 1), rnorm(n = m - 2, mean = 1))
y = rmvnorm(n = n, mean = rep(0, m), sigma = omega0 + tcrossprod(beta0))
# y3 = scale(y3, center = TRUE, scale = TRUE)
swap_order = sample(m)
y_swap = y[, swap_order]
```

```
beta0
```

```
##           [,1]      [,2]
## [1,] 0.6013929 0.000000
## [2,] 1.0408822 3.025225
## [3,] 2.0033519 1.306401
```

Note that the columns of y permuted so that we can test if the result of using this prior does indeed result in the invariance mentioned in the paper. Observe below that the 2nd and 3rd columns of y_{swap} . **Also, note that we do not center/scale y .**

```
head(y)
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.4946845 2.7994692 1.9985334
## [2,] -2.4884341 2.8090091 -4.1084074
## [3,] -0.7280545 -3.3073054 -2.4471979
## [4,] -1.1142781 -2.9187178 -3.3635541
## [5,] 0.2052317 1.1813775 0.2854175
## [6,] 0.8002731 0.2698984 2.6160572
```

```
head(y_swap)
```

```
##           [,1]      [,2]      [,3]
## [1,] 2.7994692 1.9985334 0.4946845
```

```
## [2,]  2.8090091 -4.1084074 -2.4884341
## [3,] -3.3073054 -2.4471979 -0.7280545
## [4,] -2.9187178 -3.3635541 -1.1142781
## [5,]  1.1813775  0.2854175  0.2052317
## [6,]  0.2698984  2.6160572  0.8002731
```

Gibbs Sampler

We start the Gibbs Sampler using the true value of β, Ω , just as a sanity check to make sure everything works in the most basic case.

```
Sigma_init = diag(omega0)
beta_init = beta0

nu = 2.2
C0 = 1
s = sqrt(.1/2.2)

thin    = 20
burnin  = 2000
seed    = 123
No.Iter = 12000    # number of iterations to run Gibbs sampler

## this gives us a net: No.Iter/thin posterior samples

## Gibbs sampler for the regular dataset: y
obj = factGibbsMod(y = y, k = k, beta_init, Sigma_init, nu, C0, s,
                  No.Iter, burnin, thin, seed)

## You are done with the
## 1000
## th iterations when k is
## 2
## You are done with the
## 2000
## th iterations when k is
## 2
## You are done with the
## 3000
## th iterations when k is
## 2
## You are done with the
## 4000
## th iterations when k is
## 2
## You are done with the
## 5000
## th iterations when k is
## 2
## You are done with the
## 6000
## th iterations when k is
## 2
## You are done with the
## 7000
```

```

## th iterations when k is
## 2
## You are done with the
## 8000
## th iterations when k is
## 2
## You are done with the
## 9000
## th iterations when k is
## 2
## You are done with the
## 10000
## th iterations when k is
## 2
## You are done with the
## 11000
## th iterations when k is
## 2
## You are done with the
## 12000
## th iterations when k is
## 2
## You are done with the
## 13000
## th iterations when k is
## 2
## You are done with the
## 14000
## th iterations when k is
## 2

```

```

## Gibbs sampler for the permuted dataset: y_swap

```

```

obj_swap = factGibbsMod(y = y_swap, k = k, beta_init, Sigma_init, nu, C0, s,
                        No.Iter, burnin, thin, seed)

```

```

## You are done with the
## 1000
## th iterations when k is
## 2
## You are done with the
## 2000
## th iterations when k is
## 2
## You are done with the
## 3000
## th iterations when k is
## 2
## You are done with the
## 4000
## th iterations when k is
## 2
## You are done with the
## 5000
## th iterations when k is
## 2

```

```

## You are done with the
## 6000
## th iterations when k is
## 2
## You are done with the
## 7000
## th iterations when k is
## 2
## You are done with the
## 8000
## th iterations when k is
## 2
## You are done with the
## 9000
## th iterations when k is
## 2
## You are done with the
## 10000
## th iterations when k is
## 2
## You are done with the
## 11000
## th iterations when k is
## 2
## You are done with the
## 12000
## th iterations when k is
## 2
## You are done with the
## 13000
## th iterations when k is
## 2
## You are done with the
## 14000
## th iterations when k is
## 2

```

Next, we reconstruct Σ using the posterior samples of β and Ω and take the mean over the samples.

```

sigma_list = form_sigma(obj, length(obj$Sigma))
mean_sigma(sigma_list)

```

```

##           [,1]      [,2]      [,3]
## [1,] 0.6592578  0.9538557  1.466456
## [2,] 0.9538557  10.8876932  6.962527
## [3,] 1.4664562  6.9625271  6.887766

```

We do the same using the results from the Gibbs sampler that took in the permuted version of y .

```

sigma_swap_list = form_sigma(obj_swap, length(obj_swap$Sigma))
mean_sigma(sigma_swap_list)

```

```

##           [,1]      [,2]      [,3]
## [1,] 10.8693383  6.941977  0.9525182
## [2,]  6.9419773  6.874875  1.4605912
## [3,]  0.9525182  1.460591  0.6563398

```

Obviously, these are different, but we find that if we re-order the columns of Σ and Σ_{swap} , we see that each of the entries is quite similar, thus giving the invariant result.

```
mean_sigma(sigma_list)[swap_order, swap_order]
```

```
##           [,1]      [,2]      [,3]
## [1,] 10.8876932  6.962527  0.9538557
## [2,]  6.9625271  6.887766  1.4664562
## [3,]  0.9538557  1.466456  0.6592578
```

Comparing parameter values with posterior means

The results above seem to indicate that there is nothing wrong with the output of the posterior samples since we're able to replicate the invariant property claimed in the paper, but I think there still are some outstanding issues.

The true value of β compared to the posterior mean of the β 's from the Gibbs sampler do not match.

```
beta0
```

```
##           [,1]      [,2]
## [1,] 0.6013929 0.000000
## [2,] 1.0408822 3.025225
## [3,] 2.0033519 1.306401
```

```
beta_list = form_beta(obj, length(obj$beta))
mean_beta(beta_list)
```

```
##           [,1]      [,2]
## [1,] 0.6823432 0.000000
## [2,] 1.4039256 2.959754
## [3,] 2.1608594 1.314898
```

Next, I checked if $\beta\beta'$ would match its posterior counterpart. These were particularly off.

```
beta0 %*% t(beta0) # true value of beta beta'
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.3616735  0.6259792  1.204802
## [2,] 0.6259792 10.2354224  6.037410
## [3,] 1.2048017  6.0374103  5.720102
```

```
# using posterior samples of beta to estimate beta beta'
```

```
beta_beta_list = form_beta_beta(obj, length(obj$beta))
mean_beta_beta(beta_beta_list)
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.4699675  0.9538557  1.466456
## [2,] 0.9538557 10.7726330  6.962527
## [3,] 1.4664562  6.9625271  6.456485
```

Finally, I checked if the Σ reconstructed from the posterior samples could even accurately produce an estimate for the true Σ . They are similar, but definitely not anywhere close to perfect on the off-diagonals.

```
# true Sigma
```

```
beta0 %*% t(beta0) + omega0
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.6116735  0.6259792  1.204802
## [2,] 0.6259792 10.4854224  6.037410
```

```
## [3,] 1.2048017  6.0374103  5.970102
mean_sigma(sigma_list) # posterior estimate of Sigma
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.6592578  0.9538557  1.466456
## [2,] 0.9538557 10.8876932  6.962527
## [3,] 1.4664562  6.9625271  6.887766
```

We can compare this to the sample covariance matrix to see how good the covariance estimation is.

```
# sample covariance matrix
```

```
1/n * t(y) %*% y
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.6587114  0.9909941  1.484758
## [2,] 0.9909941 11.0913558  7.132247
## [3,] 1.4847583  7.1322471  7.006373
```

```
# sample covariance matrix
```

```
cov(y) # same as above cuz y has 0 mean
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.6599196  0.9918861  1.487075
## [2,] 0.9918861 11.1028872  7.140103
## [3,] 1.4870754  7.1401025  7.016539
```

Conclusion?

I am not too sure what to make of these results. On one hand, we are able to replicate and confirm the invariant property, and the posterior estimate of Σ is halfway reasonable. However, it seems like we cannot accurately recover β or Ω , separately. Whether or not this matters in the context of the hybrid estimator, I am unsure. However, even upon removing the center/scaling part, I am still encountering numerical issues in the EP part of the algorithm. Sample size seems large enough and the number of posterior samples is also quite large.

Before I go back into debugging potential numerical issues (which I don't think is the problem here), are there any glaringly incorrectly specified parts of the algorithm that could be the root of the problems I'm seeing?